

Received February 5, 2018, accepted February 25, 2018, date of publication February 28, 2018, date of current version March 16, 2018.

Digital Object Identifier 10.1109/ACCESS.2018.2810267

Clustering Approach Based on Mini Batch Kmeans for Intrusion Detection System Over Big Data

KAI PENG^{1,2}, (Member, IEEE), VICTOR C. M. LEUNG³, (Fellow, IEEE), AND QINGJIA HUANG⁴

¹College of Engineering, Huaqiao University, Quanzhou 362021, China

²Fujian Provincial Academic Engineering Research Centre in Industrial Intellectual Techniques and Systems, Huaqiao University, Quanzhou 362021, China

³Department of Electrical and Computer Engineering, The University of British Columbia, Vancouver, V6T 1Z4, Canada

⁴Institute of Information Engineering, Chinese Academy of Sciences, Beijing 100093, China

Corresponding author: Kai Peng (pkbupt@gmail.com)

This work was supported in part by the Quanzhou Science and Technology Project under Grant 2015Z115, in part by the Scientific Research Foundation of Huaqiao University under Grant 14BS316, in part by the College Students Innovation and Entrepreneurship Project under Grant 201710385023, and in part by the Education Scientific Research Project for Middle-age and Young Teachers of Fujian Province under Grant JZ160084. The work of K. Peng was supported by the China Scholarship Council Award for one year's research abroad at The University of British Columbia, Vancouver, Canada.

ABSTRACT Intrusion detection system (IDS) provides an important basis for the network defense. Due to the development of the cloud computing and social network, massive amounts of data are generated, which inevitably brings much pressure to IDS. And therefore, it becomes crucial to efficiently divide the data into different classes over big data according to data features. Moreover, we can further determine whether one is normal behavior or not based on the classes information. Although the clustering approach based on K -means for IDS has been well studied, unfortunately directly using it in big data environment may suffer from inappropriateness. On the one hand, the efficiency of data clustering needs to be improved. On the other hand, differ from the classification, there is no unified evaluation indicator for clustering issue, and thus, it is necessary to study which indicator is more suitable for evaluating the clustering results of IDS. In this paper, we propose a clustering method for IDS based on Mini Batch K -means combined with principal component analysis. First, a preprocessing method is proposed to digitize the strings and then the data set is normalized so as to improve the clustering efficiency. Second, the principal component analysis method is used to reduce the dimension of the processed data set aiming to further improve the clustering efficiency, and then mini batch K -means method is used for data clustering. More specifically, we use K -means++ to initialize the centers of cluster in order to avoid the algorithm getting into the local optimum, in addition, we choose the Calsski Harabasz indicator so that the clustering result is more easily determined. Compared with the other methods, the experimental results and the time complexity analysis show that our proposed method is effective and efficient. Above all, our proposed clustering method can be used for IDS over big data environment.

INDEX TERMS IDS, big data, clustering, principal component analysis, mini batch Kmeans.

I. INTRODUCTION

Intrusion Detection System (IDS) can discover the malicious activities and irregularities in the network and provide an important basis for network defense [1]–[3]. Due to the development of the cloud computing, social network, as well as mobile cloud computing, IDS has become even more important than before [4]–[6]. Meanwhile, network traffic generated by various devices and hosts are exponentially rising [7]–[12]. In 2013, a total of 1000 EB traffic was

generated globally [13]. It is estimated that worldwide network traffic will reach 8.6 ZB by 2018 according to Cisco [14]. This inevitably brings much burden to IDS. And thus, how to efficiently divide the data into different classes over big data in term of data features becomes much more and more important. Additionally, we can further determine which one is normal behavior or not based on the above classes information. Data mining is an intelligent data analysis technique that finds useful knowledge from

big data [15], [16]. Data mining method for the first time was introduced to intrusion detection in 1998 [17], and then many researchers engaged in this issue. In general, research of IDS based on data mining is mainly focused on two aspects, thus clustering and classification. More specifically, there is no label for each data record in the initial dataset for clustering issue, and the object of the clustering algorithm is to put similar data records in the same class. That means the behavior of the packet will be marked as normal class or abnormal one according to the characteristics of the data, so as to achieve the purpose of clustering data. Differ from the former one, classification is the process of mining from a dataset which has been clustered. That means each data record has a label in the dataset. i.e., for a given new data record, the purpose of the classification algorithm is to determine which class the given data record belongs to. Both the two issues are different but are indispensable parts of IDS. In this study, we mainly concentrate on the research of the former one. Overall, the main clustering algorithms can be divided into the following categories [18], thus demarcation methods [19]–[23], hierarchical methods [24], [25], density-based methods [26]–[28], as well as grid-based methods [29], [30]. Moreover, we mainly focus on the application of Kmeans over IDS.

The literature [20] proposes a cascaded algorithm over Kmeans and C4.5 algorithms for anomaly detection in computer network. Since the Kmeans clustering results are more affected by the initial of clustering centers. The literature [21] proposes using improved IDS Kmeans algorithm to solve this issue. Unfortunately, they fail to consider the efficiency of clustering. The literature [22] proposes using the Apache Spark for IDS in big data environment. Although they consider the efficiency of clustering, unfortunately we cannot effectively determine which K-value is the best from the experimental results presented by them. And therefore, it is also crucial to choose the appropriate evaluation indicator. The literature [23] proposes using Kmeans clustering algorithm over Oracle Database 10 g and 1000 clusters are made. To the best of our knowledge, a good clustering algorithm does not mean that the more clusters, the better. In addition, the experiment in [23] is only presented over 10% KDDCUP99 dataset [31]. And thus, we cannot determine the performance of the algorithm over full dataset. Above all, both the initial of clustering K-value, the efficiency of clustering, as well as the evaluation of clustering results are important for IDS clustering method.

In order to address the above issue, based on Mini Batch Kmeans clustering algorithm [32], we propose an IDS clustering method named PMBKM (Mini Batch Kmeans with Principal Component Analysis) over Anaconda [33]. Firstly, a preprocessing method is proposed to digitize the strings and then the dataset is normalized for the purpose of improving the clustering efficiency. Secondly, Principal Component Analysis (PCA) [34] method is used to reduce dimensionality on the processed dataset in order to further improve the clustering efficiency for the following step. Thirdly,

Mini Batch Kmeans is used for the data clustering. More specifically, we use Kmeans++ [35] to initialize the centers of cluster in order to avoid the algorithm getting into the local optimum, in addition, we choose the Calsski Harabasz (*CH*) indicator [36] so that the clustering results can be more easily determined. In addition, compared with Kmeans method (KM), Kmeans with PCA method (PKM), and Mini Batch Kmeans with PCA method (MBKM), as well as the literature [22] and [23], the experimental results and the time complexity analysis show that our proposed method is effective and efficient.

Our contributions in this study can be summarized as follows. Firstly, we propose using Mini Batch Kmeans with PCA (PMBKM) for IDS clustering, the experimental result shows that the clustering efficiency has been greatly improved. Secondly, we use Kmeans++ to initialize the cluster centers so as to effectively avoid the algorithm getting into local optimum and further ensure the effectiveness of clustering results. Last but not the least, we consider *CH* indicator for the evaluation of clustering results. The experimental results show that it is easy to determine which K-value is the best one. Last but not the least, not only the 10% dataset, but also the full dataset of KDDCUP99 are tested in this study. Above all, our proposed PMBKM can be used for IDS clustering over big data.

The rest of the paper is organized as follows. In Section II, the preliminaries are introduced. The main content of Mini Batch Kmeans method, Principal Component Analysis method, as well as the evaluation indicator is presented. Section III specifies Mini Batch Kmeans with Principal Component Analysis (PMBKM) for IDS. Each step in this system and relevant algorithms are described. The experiment and discussion are described in Section IV. Section V presents the related work. Finally, we conclude our work and describe the future work in Section VI.

II. PRELIMINARIES

In this section, we mainly introduce the relevant theory. The Mini Batch Kmeans method is firstly introduced in section A, followed by the introduction of Principal Component Analysis in section B. Finally, the evaluation indicator is introduced in section C.

A. MINI BATCH KMEANS METHOD

For given dataset $T = \{x_1, x_2, \dots, x_p\}$, $x_i \in \mathbb{R}^{m \times n}$ x_i represents a network record which is an n -dimensional real vector. m indicates the number of records contained in the dataset T . The object of clustering problem is to find the set C of cluster centers $c \in \mathbb{R}^{m \times n}$ so as to minimize over the dataset T of records $c \in \mathbb{R}^{m \times n}$ the following function.

$$\min \sum_{x \in T} \|f(C, x) - x\|^2 \quad (1)$$

Where, $f(C, x)$ returns the closest cluster center $c \in C$ to record x . $|C| = K$ and K is the number of clusters we want to find. To initialize the cluster centers, K records are randomly

selected by using Kmeans++ [35], and the cluster centers C are set to be equal to the values of these ones. When the amount of data become very big, the convergence rate of original Kmeans will be dropped significantly. An improved Kmeans method named Mini Batch Kmeans [32] is proposed. Differ from Kmeans [37], this one does not use all the data records in the dataset each time, but select a subset of records randomly from the dataset, and therefore greatly reduces the clustering time, and overall reduces the convergence time.

B. PRINCIPAL COMPONENT ANALYSIS METHOD

When dataset has multiple features, while many of these features are related, and thus we can speed up the learning of clustering algorithms by downscaling the data. In this study, we use Principal Component Analysis (PCA) ([34], [38], and [39]) for the dimension reduction. The main idea of the PCA is to find a direction vector and project the original dataset onto it. The object of PCA is to minimize the mean square error of the projection. Besides the dimensionality reduction of the given dataset, another advantage is no parameter limitation of PCA. That means there is no need to set parameters manual during the calculation of PCA and the final result is data-related and user-independent. For given dataset T , the object of PCA is reducing n features to d dimensions. The corresponding formulas can be expressed as follows.

1) For each data x in $T_{m \times n}$, calculate the mean of each column in $T_{m \times n}$. The first step is mean centering, thus subtracting the column mean from each record and the processed data $T_{Adjust(m \times n)}$ is obtained.

2) Calculate the covariance matrix of $T_{Adjust(m \times n)}$ and denote as C .

Let

$$c_{ij} = Cov(x_i, x_j) = E\{[x_i - E(x_i)][x_j - E(x_j)]\} \quad (2)$$

Where $i, j = 1, 2, \dots, n$

And then the covariance matrix can be expressed as follows.

$$C_{n \times n} = \begin{bmatrix} c_{11} & c_{12} & \cdots & c_{1n} \\ c_{21} & c_{22} & \cdots & c_{2n} \\ \vdots & \vdots & \dots & \vdots \\ c_{n1} & c_{n2} & \cdots & c_{nn} \end{bmatrix} \quad (3)$$

3) Calculate the eigenvalues and eigenvectors of the covariance matrix C .

4) The eigenvalues are sorted in descending order, the largest d of them are selected, and then d eigenvectors corresponding to the eigenvalues are used as column vectors to form an eigenvector matrix $EigenVectors_{n \times d}$

5) Project the sample point onto the selected eigenvector and the $FinalData_{m \times d}$ is obtained.

$$FinalData_{m \times d} = T_{Adjust(m \times n)} * EigenVectors_{n \times d} \quad (4)$$

C. EVALUATION INDICATOR

It is essential to evaluate the clustering results. Both the clustering time and Calsski Harabasz are used in this study.

1) CLUSTERING TIME t

The whole calculation time t of clustering algorithm. The shorter the better.

2) CALSSKI HARABASZ (CH)

Differ from the classification in supervised learning, we cannot directly determine which result is right or not as there is no labels in the given dataset by using precision or recall. However, we can evaluate the clustering results based on the degree of in-cluster density and the degree of inter-cluster discretization. Both the Silhouette Coefficient and CH indicator [36] are the common indicators. Compared with the former one, the calculation process of the latter one is simple and has a low overhead. And therefore we choose this one as the clustering evaluation indicator. In addition, the higher CH score is, the better the clustering result will be. In other words, the covariance of the internal data of the category is as small as possible, and the covariance between the categories is as big as possible. CH can be obtained by formula (5).

$$CH = \frac{SS_{Between}}{SS_{Within}} \times \frac{m - K}{K - 1} \quad (5)$$

Where K is the number of clusters, and m is the total number of network records, SS_{Within} represents the overall within-cluster variance and $SS_{Between}$ represents the overall between-cluster variance.

III. MINI BATCH KMEANS WITH PRINCIPAL COMPONENT ANALYSIS (PMBKM) FOR IDS

A. THE OVERVIEW OF MINI BATCH KMEANS WITH PRINCIPAL COMPONENT ANALYSIS (PMBKM)

In this section, IDS Clustering method is presented. The main steps of PMBKM are shown as follows. The PMBKM mainly consists of two steps, thus step1: Data preprocess, step 2: Mini Batch Kmeans with Principal Component Analysis for Intrusion Detection System.

B. EACH STEP OF PMBKM

Step 1 (Data Preprocess): This step mainly contains the process of string and normalization. In this study, Euclidean distance is used in Mini Batch Kmeans and thus we should make sure that the data composed of numbers. If the strings are found in the dataset, we should digitize the string by using replace function. In addition, note that the range of numbers in the given dataset may not uniform. That means the columns with large numerical value will cause the role of columns with relatively small numerical value to be ignored, however those columns may play a very important role in fact. And thus the normalization process should be performed before executing the clustering algorithm, and the object of normalization is make the characteristic data shrink in ranges of [0-1]. The **algorithm 1** firstly traverses the given dataset T and finds all the strings S in dataset T and obtain the corresponding

columns c by using find function (Line 1 to Line 3), secondly, the replace function is called to replace S with random number m (Line 6 to Line 7). And then the processed dataset T' is obtained. In addition, the normalization result T_1 is obtained by using normalization function (Line 9). Obviously, T_1 will be the input for step 2.

Algorithm 1 Data Preprocess Method

Input: Dataset T
Output: processed Dataset T_1

1. **for** $n \leftarrow 1$ to N **do**
2. **for** $m \leftarrow 1$ to N **do**
3. $(S, c) \leftarrow \text{find}(T)$
4. **end for**
5. **end for**
6. **for** $n \leftarrow 1$ to c **do**
7. $T'(S) \leftarrow \text{replace}(m)$
8. **end**
9. $T_1 \leftarrow \text{normalization}(T')$
10. **return** T_1

Step 2: Mini Batch Kmeans with Principal Component Analysis for Intrusion Detection System

Algorithm 2 illustrates the process of PMBKM. The dimension reduction data T_1 is obtained by calling PCA function (**Algorithm 3**) (Line 1). And then PMBKM obtains cluster centers k by calling Kmeansplusplus function (**Algorithm 4**) (Line 2). And then it randomly selects batch sizes and denoted as b and then form a collection of M (Line 7). In addition, it calculates the distance between each point in M and K centers and then assigns each record to the closest center (Line 8 to Line 10). And then, the center of each cluster is it recalculated. The per-center learning rates is used for fast convergence, in the manners of [32] and [40] (Line 12 to Line 15). Finally, K clusters are formed and CH is returned by calling score function (Line 19).

Algorithm 3 is implemented according to the formula (2) to (4) in **section II.C**. The PCA method mainly contain four steps. Firstly, the processed data TAdjust is obtained by calling Adjust function (Line 2). And then the covariance matrix C is obtained by using covariancematrix function (Line 3). In addition, eigenvector matrix EigenVectors is obtain by calling EigenVectors function. Finally the Dimension reduction data FinalData is obtained and returned (Line 4 to Line 5).

The Kmeansplusplus procedure of **Algorithm 4** firstly chooses a record randomly from the input dataset which names K_1 as the first cluster center (Line 2). Secondly, it computes the distance between the records x_i in T_1 and the selected cluster center K_1 by calling Euclidean_dis function, a new record K_2 will be selected as the second cluster center. Data records with larger distance values are selected with a higher probability. We use Probability function to implement this. Repeat this operation until the K centers are selected (Line 5 to Line 10).

Algorithm 2 (PMBKM)

Input: Given dataset T_1 , K (number of clusters),
 Given mini batch size b , iterations t
Maximumiter (The maximum number of iterations t)
Output: CH

1. $T'_1 \leftarrow \text{PCA}(T_1)$
2. $C \leftarrow \text{Kmeansplusplus}()$
3. // Initialize each $c \in R^{m \times n}$ by calling Kmeansplusplus procedure
4. $v \leftarrow 0$
5. $t \leftarrow 0$
6. **repeat**
7. $M \leftarrow b$ instances selected randomly from T_1
8. **for** $x \in M$ **do**
9. $d[x] \leftarrow f(C, x)$ //Cache the center closest to x
10. **end for**
11. **for** $x \in M$ **do**
12. $c \leftarrow d[x]$ // Get cached center for this x
13. $v[c] \leftarrow v[c] + 1$ // Per-center counts are updated
14. $\eta \leftarrow \frac{1}{v[c]}$ // Per-center learning rate is obtained
15. $c \leftarrow (1 - \eta)c + \eta x$ // Take gradient steps
16. **end for**
17. $t++$
18. **until** $t \leq \text{MaxIter}$
19. $CH \leftarrow \text{score}(T_1)$
20. **return** CH

Algorithm 3 PCA

Input: T , Number of principal components d
Output: *FinalData*

1. **procedure** $\text{PCA}(T, d)$
2. $T_{\text{Adjust}} \leftarrow \text{Adjust}(T)$
3. $C \leftarrow \text{covariancematrix}(T_{\text{Adjust}})$
4. $\text{EigenVectors} \leftarrow \text{EigenVectors}(C, d)$
5. $\text{FinalData} \leftarrow T_{\text{Adjust}} * \text{EigenVectors}$
6. **return** *FinalData*
7. **end procedure**

IV. EXPERIMENT AND DISCUSSION

A. EXPERIMENT ENVIRONMENT

In this section, our proposed IDS method PMBKM is tested over KDDCUP99 dataset [31]. The experiment is implemented by Python on a windows 10 Operating System, which the processor is Inter Core i7 2.7GHZ, the RAM is 16GB, and the main software platform is Eclipse and Anaconda 2.7 Scikit-learn. KDDCUP99 dataset is a 9-week network connection data collected from a simulated LAN of US Air Force. The dataset contains two types of files, one is 10% dataset named as KDDCUP.Data.10.percent.corrected and the other is the full dataset named as kddcup.data.corrected. Each data record contains forty one fixed feature attributes. Actually, there are two official documents, the first one has been labeled and the other one has no labels. We will choose the latter one for our clustering experiments.

Algorithm 4 Kmeansplusplus**Input:** Given dataset T_1 , K (number of clusters),**Output:** K_j //cluster centers

1. **procedure** Kmeansplusplus (T_1 , K)
2. $K_1 \leftarrow \text{random}(T_1)$
3. $j \leftarrow 2$
4. **repeat**
5. **for all** $i \in T_1$ **do**
6. $D(i, K_1) \leftarrow \text{Euclidean_dis}()$
7. $K_j \leftarrow \text{Probability}(\text{Distance_max})$
8. $j++$
9. **end for**
10. **until** $j = K$
11. **return** K_j
12. **end procedure**

B. THE COMPARISON EXPERIMENTS DESIGN

In step 2, four kinds of clustering methods are implemented and compared. Thus, Mini Batch Kmeans with PCA (PMBKM) (Algorithm 2), Kmeans (KM), and Kmeans method with PCA (PKM), as well as Mini Batch Kmeans (MBKM). We show a basic description of the other three methods. The process of KM is shown as follows. This algorithm firstly selects K centers from the given dataset T_1 by calling Kmeansplusplus. The procedure of Kmeansplusplus is the same as the function in Kmeansplusplus in Algorithm 4. And then it calculates the distance between each point in T_1 and K centers and then assigns each point to the nearest center. Thirdly, recalculates the center of each cluster. Finally, k clusters will be formed and CH is returned by calling score function. As the main idea of PKM is the same as KM. And therefore we only describe the difference between KM and PKM. The main difference is that PKM uses PCA function firstly (Algorithm 3). The remaining process is the same as the former one. Meanwhile, as the main idea of MPKM is the same as PMBKM. And therefore we only describe the difference between algorithm MBKM and algorithm PMBKM. The main difference is that MBKM does not use PCA function (Algorithm 3).

C. EXPERIMENTAL RESULTS AND DISCUSSION

Selecting a different value of K , we mainly conduct two groups of experiments. The first group is that $K = \{10, 20, 30, 40, 50, 60, 70, 80, 90, 100\}$ and the other one is that $K = \{2, 3, 4, 5, 6, 7, 8, 9\}$. The above four methods are compared from the perspective of clustering time t and CH . The experiments are presented over both 10% dataset and full dataset.

- 1) THE FIRST GROUP ($K = \{10, 20, 30, 40, 50, 60, 70, 80, 90, 100\}$)

We present the experiment over 10% dataset and full dataset.

a: 10% DATASET

As shown in Figure 1, MBKM and PMBKM are time guaranteed. With the increasing of K , the clustering time is

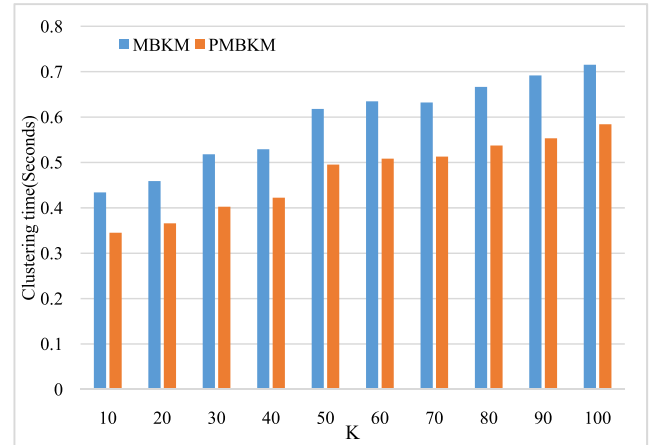


FIGURE 1. Clustering time comparison of MBKM and PMBKM over 10% dataset.

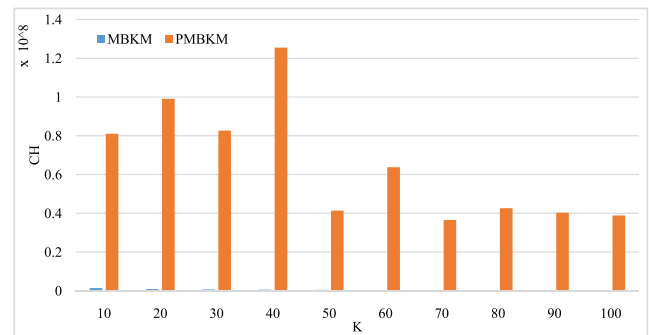


FIGURE 2. CH comparison of MBKM and PMBKM over 10% dataset.

acceptable and stable. Additionally, the clustering time of PMBKM is much shorter than MBKM. That means the formal one is more suitable for big data environment.

As shown in Figure 2, CH of MBKM has been ignorant compared with PMBKM, meanwhile, PMBKM obtains the best clustering results when K equals 40. Next, we test the experiment over full dataset.

b: FULL DATASET

As shown in Figure 3, the clustering time over full dataset is the same as the situation over 10% dataset. Both MBKM and PMBKM are effective and the latter one is much better regardless of the value of K .

As shown in Figure 4, CH of MBKM has been ignorant compared with PMBKM, meanwhile, PMBKM obtains the best clustering results when K equals 30.

c: THE EXPERIMENTAL RESULTS COMPARISON BETWEEN 10% DATASET AND FULL DATASET

We analyze the clustering time and CH for the proposed method PMBKM in both 10% dataset and full dataset. As shown in Figure 5, the clustering time increases with the increase of dataset, but the change is stable. At the same time, with the change of K , the time also has a certain change, but the change is stable.

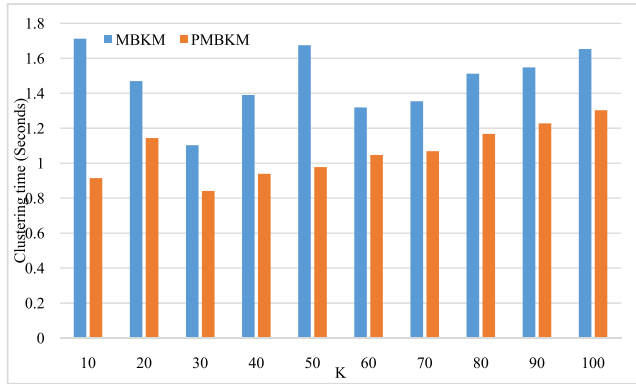


FIGURE 3. Clustering time comparison of MBKM and PMBKM over full dataset.

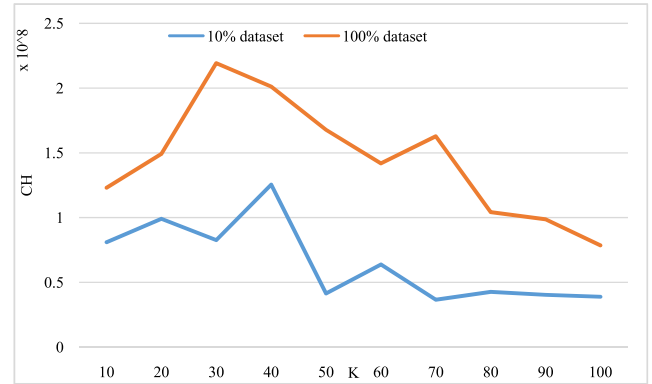


FIGURE 6. CH comparison of PMBKM over 10% dataset and full dataset.

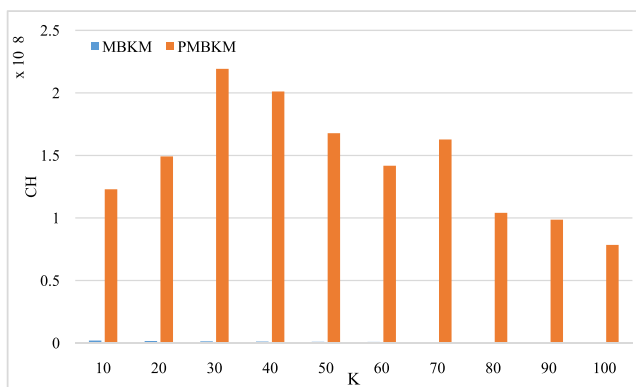


FIGURE 4. CH comparison of MBKM and PMBKM over full dataset.

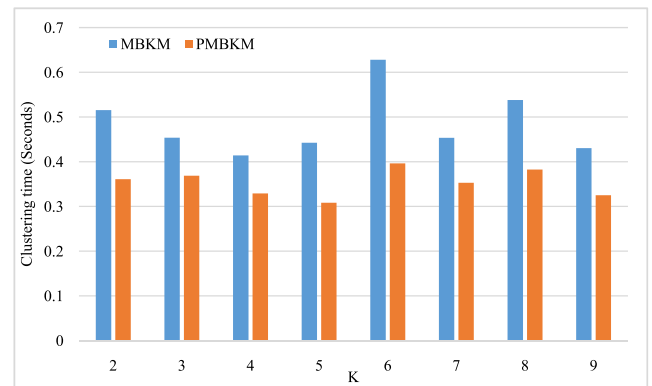


FIGURE 7. Clustering time comparison of MBKM and PMBKM over 10% dataset.

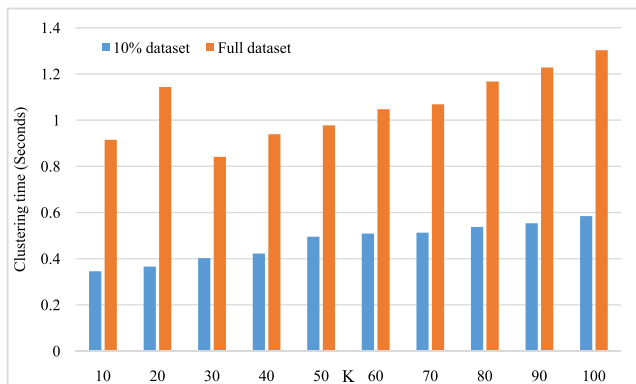


FIGURE 5. Clustering time comparison of PMBKM over 10% dataset and full dataset.

Next, we analyze CH of PMBKM method in both cases of 10% dataset and full dataset. As shown in Figure 6, the trend of the curve is similar, it starts to rise and then drop. The difference is that PMBKM obtains the best clustering result when K equals 40 over 10% dataset while K equals 3 over full dataset. That means our method is effective and can be used for different scenarios.

Overall, from the perspective of clustering time and CH , we can conclude that our proposed method PMBKM is effective and can be used for big data environment.

2) THE SECOND GROUP ($K = \{2, 3, 4, 5, 6, 7, 8, 9\}$)

Because the K value is relatively small may be more sensitive to the center value. We added a second group of experiment to test the validity of our method. Thus, test the situation that $K = \{2, 3, 4, 5, 6, 7, 8, 9\}$.

a: 10% DATASET

As shown in Figure 7, both MBKM and PMBKM are time guaranteed. And the latter one is much better regardless of the value of K . In addition, the clustering time is very stable, and thus PMBKM is very efficient.

As shown in Figure 8, CH of MBKM has been ignorant compared with PMBKM. In addition, PMBKM is increasing with the change of K value. And PMBKM obtains the best clustering result when K equals 9. And thus our method is still effective when the value of K is quite small over 10% dataset.

b: FULL DATASET

As shown in Figure 9, both MBKM and PMBKM are time guaranteed. And the latter one is much better regardless of the value of K . In addition, the clustering time is very stable, and thus PMBKM is very efficient.

As shown in Figure 10, CH of MBKM has been ignorant compared with PMBKM. In addition, PMBKM is increasing

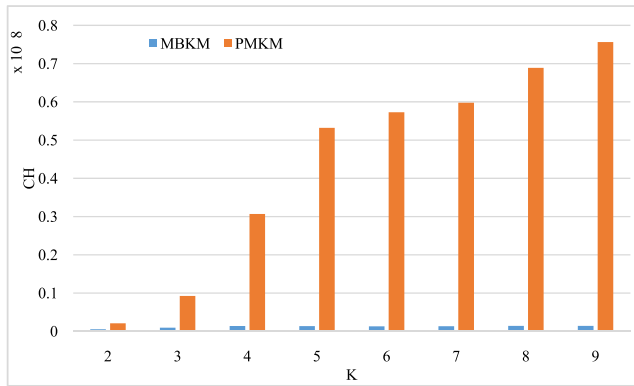


FIGURE 8. CH comparison of MBKM and PMBKM over 10% dataset.

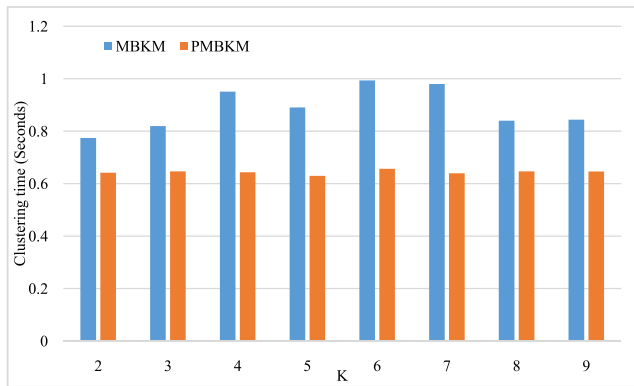


FIGURE 9. Clustering time comparison of MBKM and PMBKM over full dataset.

with the change of K -value. And PMBKM obtains the best clustering result when K equals 9. And thus our method is still effective when the value of K is quite small over full dataset.

c: THE EXPERIMENTAL RESULT COMPARISON BETWEEN 10% DATASET AND FULL DATASET

We analyze the clustering time t and CH for the proposed method PMBKM in both 10% dataset and full dataset. As shown in Figure 11, the clustering time is stable in both two situations. And thus, PMBKM is effective with the increasing of dataset.

Next, we analyze CH of this method in both cases of 10% dataset and full dataset. As shown in Figure 12, the trend of the curve is similar, CH is increasing with the change of K . PMBKM obtains the best clustering result when K equals 9 over both two kinds of dataset. Above all, from the perspective of calculation time t and CH , we can conclude that our proposed method PMBKM is efficient and effective.

d: EXPERIMENT COMPARISON OF KM, PKM, MBKM AND PMBKM OVER 10% DATASET

The four methods, KM, PKM, MBKM, as well as PMBKM are compared. Because the calculation time overhead of KM and PKM algorithms is very large, we only compare

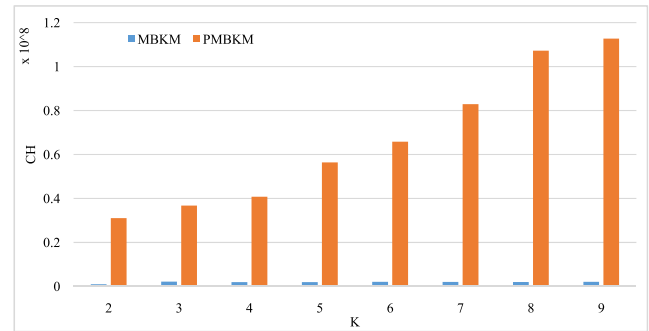


FIGURE 10. CH comparison of MBKM and PMBKM over full dataset.

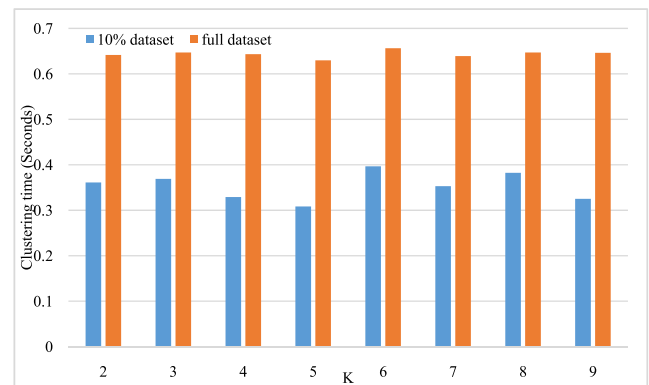


FIGURE 11. Clustering time comparison of PMBKM over 10% dataset and full dataset.

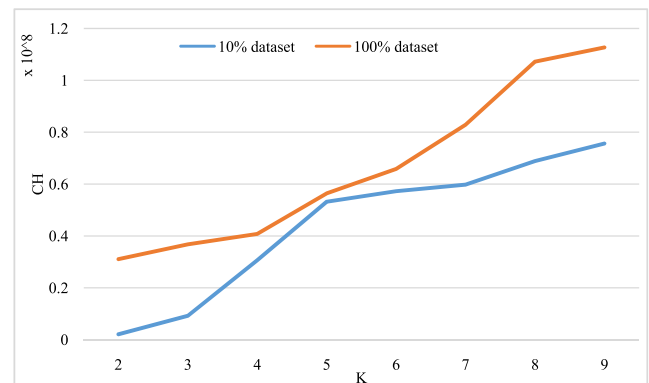


FIGURE 12. CH comparison of PMBKM over 10% dataset and full dataset.

the experimental results over 10% dataset. As shown in Figure 13, from the perspective of clustering time, our proposed PMBKM is the best. As shown in Figure 14, CH of PKM is the best, followed by PMBKM. Since PMBKM randomly selects some samples (200 samples in this study) not all the samples for calculation, the value of CH is generally lower than MBKM. However, since the value of CH is increasing, we still can conclude which K is the best one. Thus the best clustering result of PMBKM is that K equals 30. In addition, PMBKM obtain the same CH as PKM when K equals 30. Above all, our proposed PMBKM is effective.

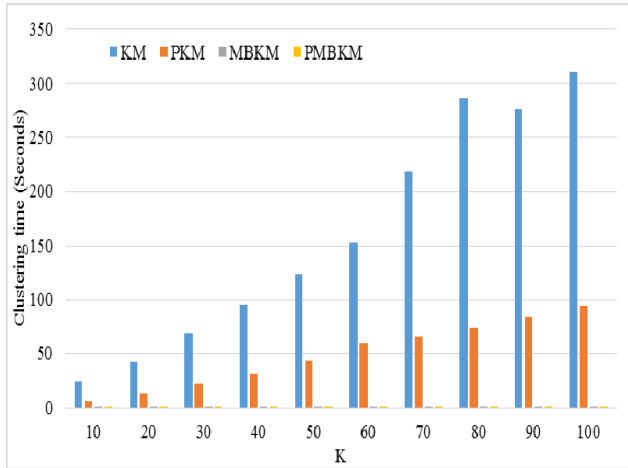


FIGURE 13. Clustering time comparison of KM, PKM, MBKM and PMBKM over 10% dataset.

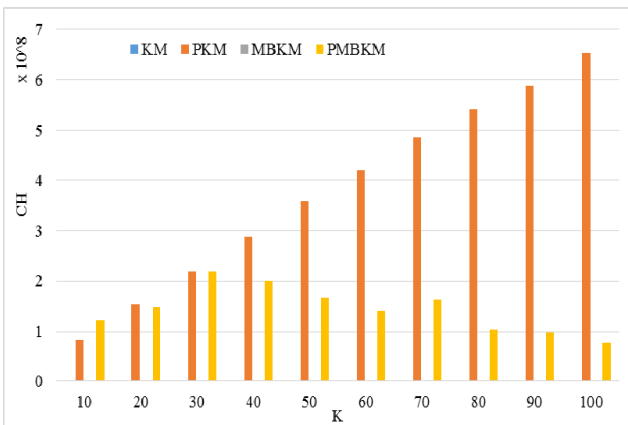


FIGURE 14. CH comparison of KM, PKM, MBKM and PMBKM over 10% dataset.

e: DISCUSSION

Time Complexity Analysis of Each Method (PMBKM, KM, PKM, MBKM): The PMBKM combines all three algorithms of Kmeans++, PCA, and Mini Batch Kmeans. As the time complexity of Kmeans++ is $O(m)$, the time complexity of PCA is $O(n * n * d)$ and the time complexity of Mini Batch Kmeans is $O(b * t * K)$. And thus the time complexity of PMBKM is $O(m + n * n * d + b * t * K)$. The time complexity of MBKM is $O(m + b * t * K)$. As shown in the Section III, m is the number of data records, n is the dimension of dataset and d is the number of principal components. b is the given mini batch size and t is the number of iteration. K is the number of clusters. Obviously, m is the smallest value among the three cost, while the value of $(b * t * K)$ is larger than $(n * n * d)$. Hence, the complexity of PMBKM and MBKM is in the same level. Based on the same principle, the time complexity of KM is $O(m + m * t * K)$, the time complexity of PKM is $O(m + n * n * d + m * t * K)$. As a result of b is much smaller than m , and thus both MBKM and PMBKM obtain the low cost. Furthermore, PMBKM is much better than MBKM as

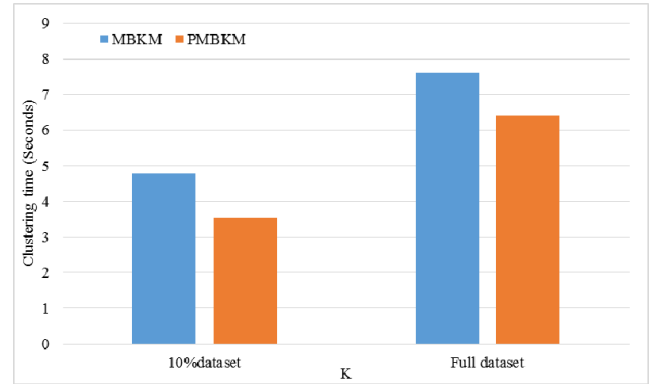


FIGURE 15. Clustering time comparison of MBKM and PMBKM over 10% dataset and full dataset.

a result of the clustering efficiency is improved after PCA process. Because the PMBKM only needs to take into account of d features instead of n features in clustering process. Moreover, both the 10% dataset and full dataset of KDDCUP99 are tested in our study, the experimental results show that the performance of PMBKM has been stable with the increasing of data number. So we can conclude that our propose method PMBKM is efficient and can be used for big data.

Discussion With Relevant Researches: In addition, Harifi et al. [22] propose using the Apache Spark for IDS in big data environments, both the 10% dataset and the full dataset are tested. Although they consider and improve the efficiency of clustering, unfortunately we cannot effectively determine which K-value clustering works best from the experimental results presented by them. Meanwhile, the clustering time in this study is much lower than that of theirs. Next, we compare PMBKM with the method in [23]. We mainly discuss the situation when K equals 1000 over both 10% dataset and the full dataset. As shown in Figure 15, both MBKM and PMBKM are time efficiency. This also proves that our proposed method PMBKM can be used in different scenarios, regardless of which value K chooses. However, we hold the opinion that there is no need to choose so big value of K . For one thing, it does not mean that the bigger the clustering value is, the larger the CH will be. In addition, as shown in Figure 6, PMBKM obtains the best clustering results when K equals 30. As shown in Figure 6 and Figure 16, CH at K equals 1000 is basically the same as K equals 30, however, the time overhead greatly increases when K equals 1000. For another, the clustering result is the reference for classification, too many clusters will reduce the accuracy of the classification.

V. RELATED WORK

IDS based on data mining is mainly concentrated in two aspects, thus clustering and classification. The initial dataset of the clustering algorithm has no labels while the classification algorithm has labels. In this study, we mainly focus on the clustering research. The clustering algorithms can be divided into the following categories, thus partitioning methods, hierarchical methods, density-based methods,

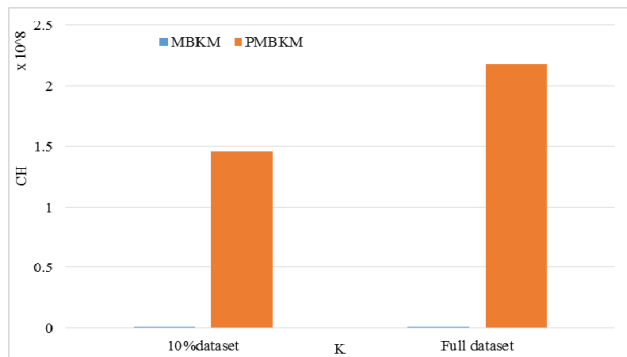


FIGURE 16. CH comparison of MBKM and PMBKM over 10% dataset and full dataset.

grid-based methods, as well as model-based methods. Clustering algorithms based on partitioning mainly include Kmeans algorithm and Kmedoids algorithm [19]. For a given dataset, the N objects contained in the dataset need to be divided into K clusters, each cluster represents a division. The partitioning methods mainly use similarity measurement function such as Euclidean distance function and angle cosine function. Differ from the partitioning methods, Hierarchical clustering method does not need to preset the number of clusters. The typical hierarchical clustering algorithms mainly contain Birch algorithm [24] and Cure algorithm [25]. In addition, there are density-based clustering algorithms which include Denclue [26], DBSCAN [27] as well as OPTICS [28]. Moreover, there are grid-based clustering algorithms, such as WaveCluster [29] and Clique [30]. These algorithms firstly divide the data space into a limited number of grids and record the data point information contained in each grid, and then present the clustering operation. On the whole, the above clustering methods are designed for solving different issues and there is no universal clustering method for solving all the problems. In this study, we mainly engage in the application of Kmeans for IDS. Muniyandi *et al.* [20] proposed using “Kmeans + C4.5” for classifying anomalous and normal activities in a computer network. Firstly, the given training examples are partitioned into K clusters by using Kmeans. And each cluster represented a density region of normal or abnormal instances. And then the decision tree was established by using C4.5 decision tree algorithm for the attacks detection. Moreover, as a result of the convergence of Kmeans is highly depending on the initial center point. In order to overcome this drawback, Liu *et al.* [21] proposed an improved Kmeans clustering algorithm. Unfortunately, the above researches did not pay attention to the clustering efficiency. Harifi *et al.* [22] proposed using Kmeans over Apache Spark Machine Learning Library, and three benchmark datasets, such as Forest Cover Type, KDDCUP99 and Internet Advertisements are tested for their experiments. Although they consider the efficiency of clustering, unfortunately we cannot effectively decide which K -value clustering works best from the experimental results presented by them. Siddiqui and Naahid [23] proposed using Kmeans clustering algorithm over ODM and 1000 clusters

are built to segment the given data records. To the best of our knowledge, a good clustering result does not mean that the more clusters are, the better the clustering results will be. In addition, the experiment in [23] is only presented over 10% KDDCUP99 dataset. Consequently, we cannot judge the performance of the algorithm over full dataset. Differ from the above researches, both the clustering efficiency and results evaluation are taken into account in this study. The Mini Batch Kmeans method is used for improving the efficiency of clustering. The Kmeans++ method is used to initialize the cluster centers so as to avoid the algorithm getting into the local optimum. Additionally, the CH indicator is chosen to evaluate the result so that it is easier to determine which K in the clustering result is best.

VI. CONCLUSION

In this study, we proposed a clustering method based on Mini Batch Kmeans with PCA (PMBKM) for Intrusion Detection System. Taking IDS classic dataset KDDCUP99 for example, both 10% dataset and full dataset are tested. Firstly, we preprocess the given dataset and then the PCA method is used to reduce the dimension so as to improve the clustering efficiency. Additionally, the Mini Batch Kmeans method is used for the clustering of the processed dataset. Compared with Kmeans (KM), Kmeans with PCA (PKM), as well as Mini Batch Kmeans (MBKM), the experimental results show that our proposed PMBKM is effective and efficient. Above all, PMBKM can be used for intrusion detection system over big data environment. In our future work, we will engage in the research of clustering method over fog computing.

REFERENCES

- [1] “Computer security threat monitoring and surveillance,” J.P. Anderson Comp., Fort Washington, PA, USA, Tech. Rep., Apr. 1980, vol. 17.
- [2] D. E. Denning, “An intrusion-detection model,” *IEEE Trans. Softw. Eng.*, vol. SE-13, no. 2, pp. 222–232, Feb. 1987.
- [3] A. Milenkoski, M. Vieira, S. Kounev, A. Avritzer, and B. D. Payne, “Evaluating computer intrusion detection systems: A survey of common practices,” *ACM Comput. Surv.*, vol. 48, no. 1, pp. 1–41, Sep. 2015.
- [4] K. Peng, R. Lin, B. Huang, H. Zou, and F. Yang, “Link importance evaluation of data center network based on maximum flow,” *J. Internet Technol.*, vol. 18, no. 1, pp. 23–31, Jan. 2017.
- [5] X. Xu *et al.*, “Data placement for privacy-aware applications over big data in hybrid clouds,” *Secur. Commun. Netw.*, vol. 2017, Nov. 2017, Art. no. 2376484, doi: <https://doi.org/10.1155/2017/2376484>
- [6] L. Qi *et al.*, “Structural balance theory-based e-commerce recommendation over big rating data,” *IEEE Trans. Big Data.*, Sep. 2016, doi: [10.1109/TBDDATA.2016.2602849](https://doi.org/10.1109/TBDDATA.2016.2602849).
- [7] H. Zhou, C. M. V. Leung, C. Zhu, S. Xu, and J. Fan, “Predicting temporal social contact patterns for data forwarding in opportunistic mobile networks,” *IEEE Trans. Veh. Technol.*, vol. 66, no. 11, pp. 10372–10383, Nov. 2017.
- [8] S. G. Wang, A. Zhou, M. Yang, L. Sun, and C.-H. Hsu, “Service composition in cyber-physical-social systems,” *IEEE Trans. Emerg. Topics Comput.*, Feb. 2017, doi: [10.1109/TETC.2017.2675479](https://doi.org/10.1109/TETC.2017.2675479).
- [9] L. Qi, W. Dou, Y. Zhou, J. Yu, and C. Hu, “A context-aware service evaluation approach over big data for cloud applications,” *IEEE Trans. Cloud Comput.*, Dec. 2015, doi: [10.1109/TCC.2015.2511764](https://doi.org/10.1109/TCC.2015.2511764).
- [10] H. Zhou, H. Wang, X. Li, and V. Leung, “A survey on mobile data offloading technologies,” *IEEE Access.*, vol. 6, no. 1, pp. 5101–5111, Jan. 2018.
- [11] R. Deng, P. Zhuang, and H. Liang, “CCPA: Coordinated cyber-physical attacks and countermeasures in smart grid,” *IEEE Trans. Smart Grid.*, vol. 8, no. 5, pp. 2420–2430, Sep. 2017.

- [12] H. Zhou, S. Xu, D. Ren, C. Huang, and H. Zhang, "Analysis of event-driven warning message propagation in vehicular ad hoc networks," *Ad Hoc Netw.*, vol. 55, pp. 87–96, Feb. 2017.
- [13] M. Wall, "Big Data: Are you ready for blast-off," BBC Bus. News, Tech. Rep., Mar. 2014.
- [14] "Forecast and Methodology, 2013–2018," Cisco Syst., San Jose, CA, USA, White Paper, 2013.
- [15] X.-W. Chen and X. Lin "Big data deep learning: Challenges and perspectives," *IEEE Access*, vol. 2, pp. 514–525, 2014.
- [16] X. Wu, X. Zhu, G.-Q. Wu, and W. Ding, "Data mining with big data," *IEEE Trans. Knowl. Data Eng.*, vol. 26, no. 1, pp. 97–107, Jan. 2014.
- [17] W. Lee and S. J. Stolfo, "Data mining approaches for intrusion detection," in *Proc. 7th USENIX Secur. Symp.*, Jan. 1998, pp. 79–93.
- [18] J.-G. Sun, J. Liu, and L. Y. Zhao, "Clustering algorithms research," *J. Softw.*, vol. 19, no. 1, pp. 48–61, Jan. 2008.
- [19] H.-S. Park and C.-H. Jun, "A simple and fast algorithm for k-medoids clustering," *Expert Syst. Appl.*, vol. 36, no. 2, pp. 3336–3341, 2009.
- [20] A. P. Muniyandi, R. Rajeswari, and R. Rajaram, "Network anomaly detection by cascading K-means clustering and C4.5 decision tree algorithm," *Procedia Eng.*, vol. 30, pp. 174–182, Jan. 2012.
- [21] H. C. Liu, X. N. Hou, and Z. Yang, "Design of intrusion detection system based on improved K means algorithm," *Comput. Technol. Devices*, vol. 26, no. 1, Jan. 2016.
- [22] S. Harifi, E. Byagowi, and M. Khalilian, "Comparative study of apache spark MLlib clustering algorithms," in *Proc. Int. Conf. Data Mining Big Data*, Jun. 2017, pp. 61–73.
- [23] M. K. Siddiqui and S. Naahid, "Analysis of KDD CUP 99 dataset using clustering based data mining," *Int. J. Data Theory Appl.*, vol. 6, no. 5, pp. 23–34, Jun. 2013.
- [24] S. Madan and K. J. Dana, "Modified balanced iterative reducing and clustering using hierarchies (m-BIRCH) for visual clustering," *Pattern Anal. Appl.*, vol. 19, no. 4, pp. 1023–1040, Nov. 2016.
- [25] S. Guha, R. Rastogi, and K. Shim, "CURE: An efficient clustering algorithm for large databases," *ACM SIGMOD Rec.*, vol. 27, no. 2, pp. 73–84, 1998.
- [26] A. Hinneburg and D. A. Keim, "An efficient approach to clustering in large multimedia databases with noise," in *Proc. 4th. Int. Conf. Knowl. Discovery. Data Mining*, vol. 98, Aug. 1998, pp. 58–65.
- [27] M. Ester, H. P. Kriegel, J. Sander, and X. Xu, "A density-based algorithm for discovering clusters in large spatial databases with noise," in *Proc. 2nd Int. Conf. Knowl. Discovery Data Mining*, Portland, OR, USA, Aug. 1996, vol. 96, no. 34, pp. 226–231.
- [28] M. Ankerst, M. M. Breunig, H.-P. Kriegel, and J. Sander, "OPTICS: Ordering points to identify the clustering structure," *ACM SIGMOD. Rec.*, vol. 28, no. 2, pp. 49–60, Jun. 1999.
- [29] G. Sheikholeslami, S. Chatterjee, and A. Zhang, "Wavecluster: A multi-resolution clustering approach for very large spatial databases," in *Proc. 24th. Int. Conf. Very Large Databases*, vol. 98, Aug. 1998, pp. 428–439.
- [30] D. Medeiros, "Process mining based on clustering: A quest for precision," in *Proc. Int. Conf. Bus. Process Manage.*, vol. 4928, Sep. 2007, pp. 17–29.
- [31] *KDD-CUP99*. [Online]. Available: <http://kdd.ics.uci.edu/databases/kddcup99/kddcup99.html>
- [32] D. Sculley, "Web-scale k-means clustering," in *Proc. 19th Int. Conf. World Wide Web*, Apr. 2010, pp. 1177–1178.
- [33] *Scikit-Learn*. Accessed: Dec. 2017. [Online]. Available: <http://scikit-learn.org/stable/index.html>
- [34] N. Halko, P. G. Martinsson, and J. A. Tropp, *Finding Structure With Randomness: Stochastic Algorithms for Constructing Approximate Matrix Decompositions*. Accessed: Dec. 2017. [Online]. Available: <http://resolver.caltech.edu/CaltechAUTHORS:20111012-111324407>
- [35] *K-means++*. Accessed: Dec. 2017. [Online]. Available: <https://en.wikipedia.org/wiki/K-means%2B%2B>
- [36] T. Caliński and J. Harabasz, "A dendrite method for cluster analysis," *Commun. Stat., Theory Methods*, vol. 3, no. 1, pp. 1–27, 1974.
- [37] D. Arthur and S. Vassilvitskii, "How slow is the k-means method?" in *Proc. 22nd. Annu. Symp. Comput. Geometry*, Jun. 2006, pp. 144–153.
- [38] M. E. Tipping and C. M. Bishop, "Mixtures of probabilistic principal component analyzers," *Neural Comput.*, vol. 11, no. 2, pp. 82–443, Feb. 1999.
- [39] P.-G. Martinsson, V. Rokhlin, and M. Tytgert, "A randomized algorithm for the decomposition of matrices," *Appl. Comput. Harmon. Anal.*, vol. 30, no. 1, pp. 47–68, 2011.
- [40] L. Bottou and Y. Bengio, "Convergence properties of the k-means algorithms," in *Proc. Adv. Neural. Inf. Process. Syst.*, 1995, pp. 585–592.



KAI PENG (M'18) received the Ph.D. degree in computer science from the State Key Laboratory of Network and Switching Technology, Beijing University of Posts and Telecommunications, Beijing, China. He is currently an Assistant Professor with the College of Engineering, Huaqiao University, China. He has co-authored over 10 journal/conference papers. His research interests include cloud computing, big data, and mobile cloud computing. He has been awarded a scholarship under the Stated Scholarship Fund to work as a Visiting Faculty with the Department of Electrical and Computer Engineering, The University of British Columbia, Vancouver, Canada, from 2017 to 2018. He is a CCF member and an ACM member. He was the TPC Member of IEEE SCS 2017 and a reviewer of China Communication and *Security and Communication Networks*.



VICTOR C. M. LEUNG (S'75–M'89–SM'97–F'03) received the B.A.Sc. degree (Hons.) in electrical engineering from The University of British Columbia (UBC) in 1977, and was awarded the APEBC Gold Medal as the Head of the graduating class in the Faculty of Applied Science. He attended graduate school at UBC on a Canadian Natural Sciences and Engineering Research Council Postgraduate Scholarship and received the Ph.D. degree in electrical engineering in 1982.

From 1981 to 1987, he was a Senior Member of Technical Staff and a Satellite System Specialist at MPR Teltech Ltd., Canada. In 1988, he was a Lecturer with the Department of Electronics, The Chinese University of Hong Kong. He returned to UBC as a Faculty Member in 1989, and currently holds the position of a Professor and TELUS Mobility Research Chair in advanced telecommunications engineering with the Department of Electrical and Computer Engineering. He has co-authored over 1000 journal/conference papers, 37 book chapters, and co-edited 13 book titles. Several of his papers had been selected for best paper awards. His research interests include the broad areas of wireless networks and mobile systems.

Dr. Leung is a Fellow of the Royal Society of Canada, the Engineering Institute of Canada, and the Canadian Academy of Engineering. He was a recipient of the IEEE Vancouver Section Centennial Award, 2011 UBC Killam Research Prize, and the 2017 Canadian Award for Telecommunications Research. He co-authored papers that won the 2017 IEEE ComSoc Fred W. Ellersick Prize and the 2017 IEEE Systems Journal Best Paper Award. He is serving on the Editorial Boards of the IEEE TRANSACTIONS ON GREEN COMMUNICATIONS AND NETWORKING, the IEEE TRANSACTIONS ON CLOUD COMPUTING, the IEEE ACCESS, *Computer Communications*, and several other journals, and has previously served on the Editorial Boards of the IEEE JOURNAL ON SELECTED AREAS IN COMMUNICATIONS–Wireless Communications Series and Series on Green Communications and Networking, the IEEE TRANSACTIONS ON WIRELESS COMMUNICATIONS, the IEEE TRANSACTIONS ON VEHICULAR TECHNOLOGY, the IEEE TRANSACTIONS ON COMPUTERS, the IEEE WIRELESS COMMUNICATIONS LETTERS, and *Journal of Communications and Networks*. He has guest-edited many journal special issues, and provided leadership to the organizing committees and technical program committees of numerous conferences and workshops. He was a Distinguished Lecturer of the IEEE Communications Society. He is a registered Professional Engineer in the Province of British Columbia, Canada.



QINGJIA HUANG received the Ph.D. degree in computer science from the State Key Laboratory of Network and Switching Technology, Beijing University of Posts and Telecommunications, Beijing, China. He is currently an Assistant Researcher with the Institute of Information Engineering, Chinese Academy of Sciences. He has co-authored over 10 journal/conference papers. His research interests include cloud computing security and malware analysis.

...